PaperCast: An Interactive, Evidence-Grounded Audio Learning Assistant

Project Workbook

By

Asha Vinodhini Vijayan
Gayathri Rukmadhavan
Manasa Sadhu
Sreya Somisetty
September 24, 2026
Advisor: Professor Chandrasekar Vuppalapati

Chapter 1. Literature Search, State of the Art

1.1 Literature Search

The literature review for PaperCast surveyed the academic foundations needed to convert complex scientific and technical documents into reliable, audience-adaptive audio. It covered nine core research areas: text-to-speech and speech synthesis, retrieval-augmented generation, tool-using agents, long-document understanding, factuality and faithfulness, controllable summarization, multimodal document analysis, and conversational audio generation.

Sources were collected from Google Scholar, the SJSU Library databases, IEEE Xplore, the ACM Digital Library, the ACL Anthology, SpringerLink, ScienceDirect, and arXiv. Search queries combined key terms such as retrieval-augmented generation, agentic workflows, tool-using language models, long-document summarization, factuality in abstractive summarization, faithful summarization, multimodal document understanding, audience-adaptive generation, text-to-speech, and conversational podcast generation. Peer-reviewed conference and journal papers were given priority, supplemented by recent preprints and official industry documentation.

Research area	Academic findings and techniques	Relevance to PaperCast
Retrieval-augmented generation	RAG combines language generation with retrieval from an external knowledge source. Retrieved passages provide evidence for generated responses (Lewis et al., 2020).	Retrieve source passages and attach document locations to generated claims.
Tool-using agents	Agent systems combine language-model decisions with external actions such as retrieval, search, observation, and tool execution (Yao et al., 2023).	Divide PaperCast into bounded stages instead of using one unconstrained prompt.

Agent evaluation	AgentBench demonstrates that agents should be evaluated on interactive tasks, tool use, planning, and task completion rather than text quality alone (Liu et al., 2023).	Evaluate both the quality of the final audio and the reliability of the intermediate workflow.
Long-document understanding	Research addresses chunking, hierarchical summarization, section-aware retrieval, context management, and iterative refinement.	Preserve relationships among the abstract, methods, findings, limitations, figures, tables, and conclusion.
Factuality and faithfulness	A generated summary may be fluent but unsupported or may change the meaning of the source (Maynez et al., 2020).	Verify claims, numerical values, named entities, and qualifiers before audio generation.
Summarization evaluation	Summarization quality is evaluated using dimensions such as coherence, consistency, fluency, and relevance (Fabbri et al., 2021).	Evaluate whether PaperCast audio is accurate, understandable, complete, and appropriate for its selected mode.
Multimodal document understanding	Document understanding research increasingly considers text, layout, tables, charts, figures, and equations together.	Deliver spoken explanations of figures, charts, and equations that plain text cannot convey.
Conversational learning	Dialogue-based systems can support clarification, follow-up questions, and explanations at different levels of complexity.	Allow listeners to ask questions, request simpler explanations, and explore topics interactively.

RAG is an important academic foundation for PaperCast because it allows a language model to use evidence from the uploaded document rather than relying entirely on information stored in its parameters. However, the literature also shows that retrieval alone does not guarantee factual output. A model may retrieve relevant information and still misinterpret it, combine unrelated passages, or introduce unsupported details. Therefore, PaperCast treats retrieval as one part of a larger workflow that includes claim verification and revision.

Research on tool-using agents provides a foundation for PaperCast's orchestration approach. ReAct demonstrates how language models can combine decision-making with external actions such as retrieval and observation. PaperCast applies this concept through a controlled workflow in which each stage produces a defined artifact. Document ingestion produces structured sections, planning produces an outline and claim targets, generation produces an evidence-linked dialogue, verification produces findings, and revision produces an updated script.

Long-document understanding is particularly important for academic papers because important information is distributed across multiple sections and document elements. A simple summary may mention the main topic while omitting the study design, limitations, assumptions, or conditions under which the findings apply. PaperCast therefore preserves section identity, page location, document hierarchy, and relationships between text and visual elements.

The literature on factuality and faithfulness establishes that fluency should not be treated as evidence of correctness. A summary can sound natural while changing the source's meaning or presenting a possibility as a fact. PaperCast addresses this problem through claim-level verification. The verifier checks whether each substantive claim is supported, whether numerical and named-entity details match, and whether qualifiers such as *may*, *associated with*, and *in this study* are preserved.

The literature review shows that established academic methods are available for information retrieval, agentic tool use, summarization evaluation, and factuality assessment. However, these capabilities are typically investigated independently. Few existing studies combine them into a unified system that can deliver interactive, adaptive, and source-grounded academic audio. This research gap provides the motivation for developing PaperCast.

1.2 State-of-the-Art Summary

The current state of the art relevant to PaperCast integrates agentic retrieval-augmented generation, long-context multimodal language models, claim-level factuality verification, audience-adaptive generation, and realistic conversational speech synthesis. These capabilities represent advanced research and industry developments that extend beyond conventional document summarization and basic text-to-speech systems.

Agentic Retrieval-Augmented Generation

Traditional RAG generally retrieves relevant passages and provides them to a language model for response generation. The emerging state of the art is agentic RAG, in which the system can plan a task, reformulate queries, select tools, retrieve additional evidence, evaluate intermediate results, and revise its output. Agentic systems therefore support more complex workflows than a single retrieval-and-generation step.

PaperCast applies this approach through a Plan–Execute–Evaluate–Retry workflow. The planner identifies the listener's objective and creates an explanation plan. The execution stage extracts document content, retrieves evidence, and generates a dialogue. The evaluation stage checks the output against the source. If unsupported or incomplete claims are detected, the retry stage revises the script before audio rendering.

Long-Context and Multimodal Language Models

Long-context and multimodal language models represent another leading technology. These models process lengthy documents while considering text, page layout, tables, figures, charts, equations, and captions. This is important because the meaning of an academic paper is often distributed across multiple document elements.

PaperCast uses this capability to preserve relationships between sections and visual content. For example, a result described in the text may depend on a table, a figure, or a specific limitation in the discussion section. The system should therefore identify the visual element, explain its meaning, and connect it to the surrounding text rather than treating it as an isolated image.

Claim-Level Factuality Verification

Claim-level verification is a leading technique for improving the reliability of generated content. Instead of evaluating only whether a complete response sounds reasonable, the system decomposes the output into individual claims and checks each claim against supporting evidence.

PaperCast applies claim-level verification to detect unsupported statements, contradictions, incorrect numbers, inconsistent terminology, and changes in the strength of a conclusion. For example, the source may state that a result is associated with a condition, while the generated script may incorrectly state that the condition causes the result. Such changes must be identified and corrected before the audio is generated.

Audience-Adaptive Generation

Current language-generation systems can control vocabulary, length, complexity, explanation depth, tone, and organization. This supports audience-adaptive generation, in which the same document can be explained differently to a beginner, a researcher, or a listener who wants a short overview.

PaperCast applies this technology through multiple learning modes. Student mode emphasizes definitions, examples, and background concepts. Researcher mode emphasizes methodology, results, limitations, and precise terminology. Quick Brief mode presents the most important claims in a short format. Deep Dive mode provides detailed explanations and connections. Study/Review mode supports recall and follow-up questioning.

Conversational Audio Generation

Modern speech-generation systems support multiple voices, dialogue planning, turn-taking, pauses, pronunciation control, expressive prosody, and long-form audio generation. Research prototypes such as PaperWave demonstrate how academic papers can be converted into conversational podcasts using LLM-generated scripts (Yahagi et al., 2025).

Industry systems also demonstrate the feasibility of document-to-audio learning. Google NotebookLM generates Audio Overviews in which AI-generated hosts discuss user-provided sources. Later versions introduced interactive capabilities that allow users to ask questions during an audio session. Google has also described Illuminate, an experimental system that converts research papers into short conversations between AI-generated voices.

These systems establish important industry baselines, but conversational delivery introduces a reliability risk. AI-generated hosts may add examples, transitions, or interpretations that are not present in the source. PaperCast therefore separates content generation from speech rendering. The written dialogue must first pass the verification stage before it is converted into audio.

Leading-Edge Technologies and Techniques for PaperCast

Academic techniques

1. **Agentic Retrieval-Augmented Generation (Agentic RAG)**
   Integrates information retrieval with planning, query reformulation, tool use, reflection, and iterative output refinement.
2. **Evidence-grounded generation**
   Generates responses only from retrieved source passages and links each claim to supporting evidence.
3. **Claim-level factuality verification**
   Decomposes generated content into claims and checks factual support, contradictions, numerical values, named entities, and qualifiers.
4. **Long-context document understanding**
   Processes lengthy research papers while maintaining relationships across sections, pages, methods, results, and conclusions.
5. **Multimodal document understanding**
   Interprets text, tables, figures, charts, equations, captions, and document layouts using vision-language models.
6. **Hierarchical document summarization**
   Summarizes documents at multiple levels, such as section, topic, and complete-document level.
7. **Audience-adaptive generation**
   Controls vocabulary, length, complexity, explanation depth, tone, and organization based on the listener's profile.
8. **Controllable dialogue generation**
   Controls speaker roles, turn-taking, dialogue structure, question answering, and explanation style.
9. **Reflection and evaluator–optimizer workflows**
   Uses an evaluator or critic to identify weaknesses in generated content and an optimizer to revise it.
10. **Source attribution and provenance tracking**
    Maintains links between generated claims and page, section, paragraph, table, figure, or equation locations.
11. **Interactive learning and adaptive tutoring**
    Identifies user knowledge gaps, answers follow-up questions, simplifies explanations, and addresses misconceptions.

12. **Long-form neural speech synthesis**
    Generates natural speech with appropriate pauses, pronunciation, prosody, emotion, and multi-speaker dialogue.

Industry tools and platforms

1. **Google NotebookLM / Gemini Notebook**
   Provides source-grounded question answering, Audio Overviews, document analysis, and interactive audio conversations.
2. **Google Illuminate**
   Uses long-context models to transform research papers into short audio conversations between AI-generated voices.
3. **LangGraph**
   Provides graph-based orchestration for stateful, controllable, multi-step agent workflows and agentic RAG applications.
4. **LlamaIndex**
   Supports document parsing, indexing, retrieval, RAG pipelines, agent workflows, and connections between agents and private data.
5. **Haystack**
   Provides open-source components for production RAG systems, multimodal search, document stores, agents, tools, and pipelines.
6. **ElevenLabs Text-to-Speech and Dialogue APIs**
   Supports expressive speech, multi-speaker dialogue, streaming, voice control, and natural conversational audio.
7. **Multimodal large language models**
   Models such as Gemini, GPT, Claude, and other vision-language models can analyze text, images, tables, charts, and documents.
8. **Vector databases and hybrid retrieval systems**
   Tools such as Qdrant, Milvus, Pinecone, Elasticsearch, and Weaviate support semantic search, keyword search, metadata filtering, and evidence retrieval.
9. **Document parsing and OCR platforms**
   Tools such as LlamaParse, Unstructured, Azure Document Intelligence, Google Document AI, and Amazon Textract extract text, layout, tables, and document structure.
10. **LLM evaluation and observability platforms**
    Tools such as LangSmith, Arize Phoenix, TruLens, Ragas, DeepEval, and OpenTelemetry-based systems measure retrieval quality, faithfulness, relevance, latency, and cost.

Current Gap and PaperCast's Position

The leading technologies demonstrate progress in individual parts of the PaperCast problem. Agentic RAG improves retrieval and workflow control. Long-context multimodal models improve document understanding. Claim-level verification improves reliability. Adaptive generation supports different audiences. Speech models improve the naturalness and accessibility of audio.

However, these capabilities are not consistently integrated into one end-to-end academic learning system. Existing approaches generally emphasize either document question answering, document summarization, or audio generation. PaperCast addresses this gap by combining evidence retrieval, adaptive explanation planning, claim verification, interactive questioning, multimodal document narration, topic navigation, and validated audio rendering in a single workflow.

The proposed contribution is therefore not simply another text-to-speech tool. PaperCast is designed as an evidence-grounded learning assistant in which accuracy, traceability, adaptability, and listening quality are evaluated together. This positioning distinguishes the project from systems that optimize only fluency or audio realism.

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

This paper introduced the foundational retrieval-augmented generation approach. It supports PaperCast's use of source retrieval to ground generated explanations in the original document.

[2] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

This paper presents a method for combining language-model reasoning with external actions such as retrieval and observation. It supports PaperCast's use of tool-based, multi-stage agent workflows.

[3] X. Liu et al., "AgentBench: Evaluating LLMs as agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2308.03688

AgentBench provides a framework for evaluating language models as interactive agents. It is relevant to PaperCast because the system must be evaluated on planning, tool use, interaction, and task completion in addition to final response quality.

[4] G. Mialon et al., "Augmented language models: A survey," *Trans. Mach. Learn. Res.*, 2023. [Online]. Available: https://arxiv.org/abs/2302.07842

This survey examines language models enhanced with tools, external memory, retrieval, and structured workflows. It provides academic support for PaperCast's integration of retrieval, document processing, verification, and agentic orchestration.

[5] M. Mathew, D. Karatzas, and C. V. Jawahar, "DocVQA: A dataset for VQA on document images," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2021, pp. 2200–2209. [Online]. Available: https://arxiv.org/abs/2007.00398

This paper introduces a benchmark for answering questions about document images. It is relevant to PaperCast because academic documents contain visual elements such as tables, figures, layouts, and captions that must be interpreted in addition to plain text.

[6] A. R. Fabbri, I. Li, T. She, S. Li, and D. R. Radev, "SummEval: Re-evaluating summarization evaluation," *Trans. Assoc. Comput. Linguist.*, vol. 9, pp. 391–409, 2021. [Online]. Available: https://aclanthology.org/2021.tacl-1.24/

This paper evaluates methods for assessing summaries, including coherence, consistency, fluency, and relevance. It supports PaperCast's evaluation of generated explanations beyond simple text fluency.

[7] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2020, pp. 1906–1919. [Online]. Available: https://aclanthology.org/2020.acl-main.173/

This paper examines how abstractive summaries can introduce unsupported information or change the meaning of the original source. It directly supports PaperCast's claim-verification and source-faithfulness requirements.

[8] Y. Yahagi, R. Chujo, Y. Harada, C. Han, K. Sugiyama, and T. Naemura, "PaperWave: Listening to research papers as conversational podcasts scripted by LLM," arXiv preprint arXiv:2410.15023, 2025. [Online]. Available: https://arxiv.org/abs/2410.15023

This paper presents a prototype that converts academic papers into conversational podcasts using large language models. It provides a direct research foundation for PaperCast's document-to-conversational-audio capability. Since it is an arXiv publication, it should be identified as an emerging preprint rather than a peer-reviewed publication.

[9] Google, "NotebookLM now lets you listen to a conversation about your sources," *Google Blog*, Sep. 11, 2024. [Online]. Available: https://blog.google/innovation-and-ai/products/notebooklm-audio-overviews/

This industry reference describes NotebookLM's Audio Overview feature, which converts uploaded documents into conversations between AI-generated hosts. It provides an industry baseline for source-based conversational audio while also identifying limitations related to accuracy and interaction.

[10] Google, "More ways Google is delivering on its responsible AI commitment," *Google Blog*, 2024. [Online]. Available: https://blog.google/innovation-and-ai/products/google-responsible-ai-commitment-update/

This reference describes Illuminate, an experimental Google system that uses long-context models to transform research papers into short audio conversations. It is relevant to PaperCast's state-of-the-art comparison because it demonstrates an industry implementation specifically focused on academic paper-to-audio generation.

[11] ElevenLabs, "Text-to-speech and dialogue generation," ElevenLabs, n.d. [Online]. Available: https://elevenlabs.io/text-to-speech

This industry reference describes current text-to-speech and multi-speaker dialogue capabilities, including expressive voices, streaming, and conversational audio generation. These capabilities are relevant to PaperCast's final audio-rendering stage.

Chapter 2. Project Justification

Academic papers and technical documents contain important knowledge for students, researchers, and working professionals, but their length, specialized terminology, equations, tables, figures, and dense organization can make them difficult to understand efficiently. The problem becomes more pronounced when a learner is commuting, exercising, has limited screen access, or needs an explanation at a different level of depth. Conventional text summarizers can reduce document length, but they may omit important qualifications, limitations, methods, or evidence. Text-to-speech systems improve access to written material, but they generally read generated or existing text without helping the listener understand, question, or verify the content.

PaperCast is justified as a combined accessibility, learning, and scientific-integrity solution. It is designed to transform an academic document into conversational audio while keeping the generated explanation connected to the original source. The system uses a Plan, Execute, Evaluate, and Retry workflow to coordinate document extraction, retrieval, audio generation, and verification. Rather than treating fluent audio as sufficient, PaperCast is intended to expose the supporting source content and verification status for substantive claims. When the evidence is insufficient, the system should revise, qualify, or omit the claim instead of presenting an unsupported statement as fact.

The proposed system also addresses the limitations of one-size-fits-all explanations. A listener should be able to ask questions, request a simpler or deeper explanation, navigate by topic, and trace an explanation back to the relevant document content. PaperCast is intended to make figures, tables, and equations more accessible through audio descriptions while preserving the distinction between information stated in the source and an explanation added for comprehension. These capabilities support use cases in academic learning, research onboarding, accessibility, professional development, and technical-documentation review.

The project is also justified as a research and engineering platform. Its staged workflow makes it possible to evaluate document extraction, retrieval, planning, generation, verification, revision, and audio rendering separately. This supports controlled comparisons with simpler approaches, including extractive summarization, single-pass language-model summarization, and retrieval-augmented summarization. The resulting prototype, evaluation procedures, evidence-linked artifacts, and documented limitations can support the team's Phase 1 demonstration and provide a foundation for later experimentation and a publication-oriented report.

The central expected contribution is therefore not merely another document-to-audio converter. It is an interactive audio learning assistant that combines source grounding, adaptive explanation, verification, and accessible delivery in one inspectable workflow.

Chapter 3. Project Requirements

This chapter defines the development requirements for the Phase 1 PaperCast prototype. Requirements are grouped as essential, desired, optional, and non-functional. A feature is considered within scope only when its inputs, outputs, and completion conditions are sufficiently clear for the team to implement and evaluate it. Technology names below describe the planned implementation direction; the team may substitute equivalent components if the resulting behavior and evaluation goals are preserved.

3.1 Primary User and System Actors

Actor	Role in the system
Learner or reader	Uploads or selects an academic document, chooses an explanation mode, listens to the generated audio, asks questions, and reviews evidence.
Project team or evaluator	Provides test documents, reviews generated explanations, records errors, and evaluates faithfulness, usefulness, and audio quality.
PaperCast system	Extracts document content, retrieves evidence, plans and generates an explanation, verifies claims, revises unsupported content, and renders approved audio.

3.2 Essential Functional Requirements

ID	Requirement	Description	Completion condition
FR-E1	Document upload	The system shall accept a supported academic or technical document, initially including PDF, and create a unique document record.	A test PDF is accepted, stored, and assigned an identifier.
FR-E2	Document extraction	The system shall extract text and preserve available page, section, paragraph, table, and figure references.	Extracted content can be inspected and mapped to source locations.
FR-E3	Evidence retrieval	The system shall retrieve source passages relevant to a requested topic, plan item, or generated claim.	Returned evidence includes document identifier, source location, extracted text, and retrieval information.

FR-E4	Explanation planning	The system shall create a structured plan containing the requested mode, target depth or duration, topics, and explanation objectives.	A plan is produced before dialogue or narration generation.
FR-E5	Audio script generation	The system shall generate conversational narration or dialogue grounded in retrieved document evidence.	The generated script contains traceable evidence identifiers for substantive factual claims.
FR-E6	Claim verification	The system shall identify substantive claims and check them against the source evidence, including numbers, names, dates, and important qualifiers.	Each checked claim receives a verification status and an explanation of the result.
FR-E7	Revision or rejection	The system shall revise, qualify, or remove unsupported or overconfident claims before audio rendering.	Unresolved claims are not silently presented as verified facts.
FR-E8	Audio rendering	The system shall convert an approved script into playable audio and associate the audio with the approved script version.	A user can play or download the generated audio for a successful test document.
FR-E9	Evidence presentation	The system shall allow the user to inspect the source evidence associated with an explanation or claim.	A user can move from a claim or audio segment to the corresponding source location when available.

3.3 Desired Functional Requirements

ID	Requirement	Description
FR-D1	Adaptive explanation modes	Provide distinct modes such as Student, Researcher, Quick Brief, Deep Dive, Discussion, and Study/Review, with differences in vocabulary, depth, organization, and target length.

FR-D2	Interactive questions	Allow a listener to ask a question about the document and receive an evidence-grounded answer without restarting the entire workflow.
FR-D3	Follow-up adaptation	Allow a listener to request a simpler explanation, deeper explanation, example, or clarification of a previously discussed topic.
FR-D4	Topic navigation	Allow users to select or return to a document topic, section, or planned segment.
FR-D5	Multimodal content accessibility	Generate audio descriptions or explanations for relevant figures, tables, and equations while preserving their source location.
FR-D6	Retry and recovery	Retry a failed extraction, retrieval, generation, verification, or rendering stage without unnecessarily repeating successful stages.
FR-D7	Evaluation logging	Record configuration, model or component versions, latency, retries, cost-related information when available, and evaluation results.

3.4 Optional Functional Requirements

ID	Requirement	Description
FR-O1	Multiple audio voices	Support more than one approved voice or speaking style while preserving the same verified script.
FR-O2	Exportable evidence package	Export the script, evidence map, verification report, and audio metadata as a reproducible package.
FR-O3	Saved learning sessions	Allow a user to save progress, questions, and reviewed topics for later study.
FR-O4	Additional document formats	Extend ingestion beyond PDF after the initial workflow is stable.

3.5 Non-Functional Requirements

ID	Quality attribute	Requirement
NFR-1	Faithfulness and traceability	Substantive factual claims should have evidence links, and the system should preserve numerical values, named entities, negation, and source qualifiers whenever they are relevant.
NFR-2	Usability	A first-time user should be able to upload a document, select an explanation mode, start processing, inspect verification results, and play approved audio without needing to understand the internal pipeline.

NFR-3	Reliability	The system should report failures clearly, avoid rendering unapproved scripts, and retain successful intermediate artifacts when a later stage fails.
NFR-4	Performance	The system should record stage-level and end-to-end latency and should provide progress or status information during long-running processing.
NFR-5	Reproducibility	Model settings, prompts, retrieval settings, configuration files, intermediate artifacts, and evaluation results should be versioned or logged sufficiently to reproduce an experiment.
NFR-6	Security and privacy	Secrets shall be stored outside source code, uploaded documents shall not be exposed to unauthorized users, and access to model or storage services shall use controlled configuration.
NFR-7	Accessibility	Generated audio should be intelligible, appropriately segmented, and accompanied by readable text and source references where possible.
NFR-8	Maintainability	Each major stage should have a documented input/output contract and should be testable independently of the other stages.

3.6 Phase 1 Acceptance Boundary

For Phase 1, the minimum demonstrable path is: upload a supported academic PDF, extract and index its content, retrieve relevant evidence, generate an evidence-linked explanation, verify and revise its claims, render approved audio, and expose the audio together with the script and source evidence. Interactive questions, advanced listener modes, multimodal figure or equation descriptions, saved sessions, and additional export capabilities should be treated as planned extensions unless the team confirms that they are implemented and testable within the Phase 1 schedule.

Chapter 4. Dependencies and Deliverables

Dependencies

The following dependencies are required for the development and operation of the automated PaperCast pipeline

- **Document processing capability:** A reliable PDF-processing tool is required to extract text, headings, paragraphs, tables, figures, equations, and page locations.
- **Language model access:** A structured-output language model or locally runnable model is required for planning, dialogue generation, claim extraction, and script revision.
- **Evidence retrieval system:** An embedding model and vector index are required to retrieve relevant evidence from the uploaded document.
- **Automated verification capability:** The system requires claim-matching, contradiction-detection, numerical-consistency, and named-entity checks to verify generated content against the source.
- **Text-to-speech capability:** A stable text-to-speech engine is required to convert the approved script into audio.
- **Storage and computing resources:** The system requires sufficient storage and computing resources for uploaded PDFs, document indexes, generated scripts, audio files, verification reports, and evaluation records.
- **Evaluation resources:** A representative collection of scientific and technical documents is required to test the system. A small group of human reviewers may be used separately to evaluate factual accuracy, completeness, audience fit, and audio quality. Human intervention is not required during the normal automated workflow.
- **Future cloud deployment access:** AWS access may be required in a later phase to host the application, store files, and monitor the deployed system. Initial development and testing will be performed locally.

Deliverables

The project deliverables will include:

- **Automated PaperCast application:** A working application that accepts a scientific or technical PDF, identifies the selected audience mode, and produces conversational educational audio.
- **Document ingestion module:** A component that extracts document content and preserves section, page, paragraph, table, figure, and equation metadata.
- **Evidence retrieval module:** A component that creates document embeddings, indexes source content, and retrieves evidence for planning and claim verification.
- **Audience-adaptive generation module:** A component that produces Student, Researcher, Quick Brief, Deep Dive, Discussion, and Study/Review outputs based on listener preferences.
- **Agentic processing workflow:** An automated workflow containing document understanding, planning, dialogue or narration generation, claim extraction, verification, revision, and audio rendering.
- **Evidence-linked transcript:** A generated script in which substantive claims are connected to the supporting passages in the original document.
- **Automated verification module:** A component that checks claims for factual support, numerical consistency, named-entity accuracy, terminology, negation, and preservation of source limitations.
- **Audio generation component:** A component that converts the approved script into audio segments and produces a final playable audio file.
- **Baseline implementations:** Extractive summarization, single-pass abstractive generation, and retrieval-augmented generation systems for comparison with the complete PaperCast workflow.
- **Evaluation package:** A document corpus, annotation guidelines, experiment configurations, evaluation logs, metrics, and result tables.
- **Implementation documentation:** Documentation describing the local setup, system components, configuration requirements, testing process, and known limitations.
- **Future AWS deployment package:** Deployment documentation and, in a later phase, a cloud-hosted prototype using appropriate AWS services for storage, hosting, and monitoring.
- **Research artifacts:** The final technical report and, if requested by the advisor, a publication-oriented paper or demonstration package.

Chapter 5. Project Architecture

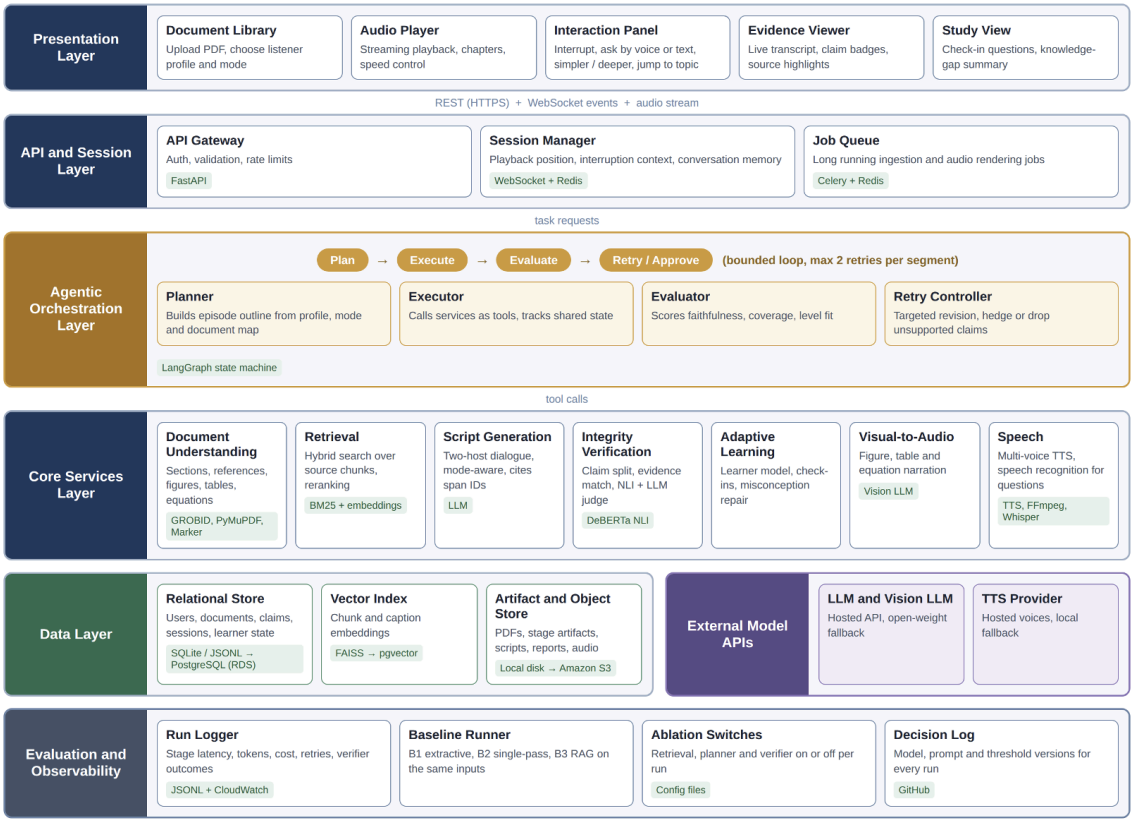


Figure 5.1. PaperCast system architecture showing functional components (bold) and planned technologies (green tags)

PaperCast uses a layered, service-oriented architecture with an agentic orchestrator at its center. Each layer only talks to the layer directly below it, and each capability in the core is a separate service with a small, typed interface. This lets us change a PDF parser, a language model, or a TTS voice without touching the rest of the system.

Presentation layer. A lightweight React (Next.js) web client where the listener uploads a paper, picks a profile (Student or Researcher) and a mode (Quick Brief, Deep Dive, Discussion, or Study/Review), and listens. While audio plays, the listener can interrupt to ask a question by voice or text, ask for a simpler or deeper explanation, jump to a topic, and see the source passage behind any claim.

API and session layer. A FastAPI server handles uploads and episode requests over REST and keeps a WebSocket open for each listening session. Redis stores the current playback position and recent conversation, so an answer to a question refers to what the listener just heard. Slow work such as parsing and audio rendering runs in the background on a Celery job queue.

Agentic orchestration layer. A LangGraph state machine runs a Plan, Execute, Evaluate, and Retry loop. The Planner builds an episode outline from the document, the listener profile, and the mode. The Executor calls the core services to write each segment. The Evaluator checks the segment for faithfulness to the source, coverage, and fit to the listener's level. The Retry Controller sends failed claims back for revision, up to two times, and then hedges or drops anything still unsupported. Each stage writes a saved, typed artifact, so a failed stage can be rerun on its own and every run can be inspected.

Core services layer. Document Understanding (GROBID, PyMuPDF, Marker) extracts sections, figures, tables, and equations with page references. Retrieval uses hybrid keyword and embedding search. Script Generation writes a two-host dialogue in which every factual sentence cites its source. Integrity Verification splits the script into claims and checks each one with an NLI model and an LLM judge. Adaptive Learning tracks what the listener understands and adds check-in questions or short fixes for misconceptions. Visual-to-Audio turns figures, tables, and equations into spoken explanations. Speech produces the voices with TTS, joins segments with FFmpeg, and transcribes spoken questions with Whisper.

Data and evaluation layers. We start with SQLite, JSON files, and a FAISS index on local machines, then move to PostgreSQL with pgvector and Amazon S3 once the prototype is stable. Hosted models sit behind adapters with open-weight fallbacks. An evaluation layer logs latency, cost, and verifier results for every run, and can run the same documents through our baselines with components switched off. This lets us measure what each part of the design contributes.

We chose this structure over a simple pipe-and-filter chain (parse, summarize, read aloud) for two reasons. A fixed chain has no place to catch and fix an unsupported claim before the listener hears it, and it cannot respond when a listener interrupts in the middle of an episode. Development and testing will run locally in Docker containers, and the same containers will later be deployed to AWS.

Chapter 8. Implementation Plan and Progress

This chapter describes how the PaperCast Phase 1 prototype will be implemented, tested, and integrated. The plan follows a staged workflow: document ingestion, evidence indexing, planning, evidence-grounded generation, claim verification, revision, and audio rendering. Each stage will produce an inspectable artifact with a documented input and output contract. This lets the team validate individual components before combining them into the complete user workflow.

8.1 Programming and Execution Environment

PaperCast will use a reproducible Python-based development environment. The environment is designed to support document processing, retrieval, orchestration, evaluation, and integration while allowing the team to test the Phase 1 workflow locally.

- **Programming language.** The implementation will use Python 3.11 or later for document processing, retrieval, orchestration, evaluation, and integration logic.
- **Application service.** A FastAPI service layer, or an equivalent Python service, will expose endpoints for document upload, processing status, evidence review, verification results, and audio access.
- **User interface.** A lightweight web client will support document selection, explanation-mode selection, processing-status inspection, evidence review, interactive questions, and audio playback.
- **Configuration.** Versioned configuration files and environment variables will store model settings and service endpoints. Secrets will remain outside the source repository.
- **Version control.** Git and a shared GitHub repository will track source code, prompts, configuration examples, documentation, experiment decisions, and implementation changes.
- **Execution modes.** The team will support a local development environment with an optional hosted model or service. This will allow repeatable local testing while preserving access to the selected language-model and text-to-speech services.

8.2 Development Tools and Component Plan

The following component plan identifies the tools under consideration and the implementation evidence expected from each component. Final tool choices will be recorded with the corresponding configuration and reproduction notes.

- **Document processing and retrieval.** PyMuPDF or Unstructured will extract text, headings, page references, and figure or table metadata. Sentence Transformers or hosted embeddings, together with SQLite or JSONL and FAISS, Chroma, or

lexical retrieval, will produce traceable document records and ranked evidence passages.

- **Planning and generation.** A structured-output language model, prompt templates, and schema validation will generate an audience-aware plan and evidence-linked narration or dialogue script.
- **Verification and revision.** Deterministic checks, together with an LLM-based or NLI-based judge where appropriate, will record claim status, findings, revised wording, or rejection decisions.
- **Audio rendering.** The selected text-to-speech engine and FFmpeg will produce segmented, playable audio tied to the approved script version.
- **Evaluation and logging.** Python scripts, notebooks, JSONL logs, and experiment configurations will capture metrics, errors, latency, retries, and reproducible results.

8.3 Example Programs, Validation, and Hardware Applicability

The team will validate the selected technologies through small, reproducible software examples before relying on them in the integrated prototype. These examples will be documented with their setup instructions, inputs, outputs, and validation results.

- **Example programs and tool validation.** The team will validate PDF extraction, retrieval, structured output, claim verification, and text-to-speech using small examples with documented inputs, outputs, and checks.
- **Current progress.** Examples are being documented and assigned while the core components are prepared. A component will be marked complete only after its example runs, its input and output format is recorded, and a validation check is completed.
- **Hardware applicability.** Hardware prototyping and simulation are not applicable because PaperCast Phase 1 is software-only. The equivalent environment is the local Python setup, web service, model APIs, and text-to-speech pipeline.

8.4 Prototype Execution Path

The first runnable prototype will follow the sequence below. The sequence is intentionally explicit so that each stage can be tested independently and traced to the final audio output.

1. Set up the pinned Python environment, dependencies, credentials, and service configuration.
2. Upload a representative academic PDF, preserve the original file, and create a traceable document record.

3. Extract and index text, headings, page references, and available figure or table metadata.
4. Select an explanation mode, generate a plan, retrieve evidence, and produce an evidence-linked script.
5. Extract and verify substantive claims, revise or reject unsupported claims, and approve the script before audio rendering.
6. Render and export playable audio together with the script, evidence references, verification results, and execution metadata.

8.5 Team Responsibilities and Current Progress

The progress descriptions below distinguish work that is being designed or prepared from work that is complete. A component will not be marked complete merely because its library or API has been selected. Completion requires a runnable example, an input/output contract, at least one validation check, and a short reproduction note.

- **Manasa Sadhu.** Research matrix, state-of-the-art sources, rubric mapping, and evaluation criteria; in progress. Evidence: source matrix, baseline definitions, and evaluation checklist.
- **Asha Vinodhini Vijayan.** Listener modes, workflow screens, and prompt contracts; in progress. Evidence: mode specification, screen flow, schemas, and one UI path.
- **Gayathri Rukmadhavan.** PDF parsing, section mapping, chunking, and retrieval; initial setup. Evidence: ingestion example, indexed sample document, and retrieval tests.
- **Sreya Somisetty.** Planning, generation, claim extraction, verification, and revision; initial design. Evidence: structured artifacts, claim-checking example, revision behavior, and unit tests.
- **Shared integration.** All members contribute to API integration, text-to-speech, experiments, metrics, and quality assurance, coordinated by Manasa. Evidence: integrated prototype, audio output, evaluation command, and QA checklist.

8.6 Implementation Checkpoints

The Phase 1 implementation will be reviewed at the following checkpoints. Each checkpoint has a defined target period and an observable result.

- **Environment and scope review - September 24–30, 2026.** The team will have a pinned environment, repository structure, requirements baseline, sample-corpus plan, and agreed component interfaces.
- **First text prototype - October 1–14, 2026.** A sample PDF will be uploaded, parsed, indexed, and processed into a first text explanation.

- **Baseline and retrieval demonstration - October 15–28, 2026.** The team will demonstrate an extractive or single-pass baseline and retrieval-enabled generation with inspectable evidence.
- **Verified script prototype - October 29–November 11, 2026.** Claim extraction, verification, revision, and approval will be connected for the common path.
- **Integrated audio prototype - November 12–25, 2026.** Approved scripts will be rendered into playable audio, and basic usability and robustness checks will be recorded.
- **Experiment freeze - November 26–December 9, 2026.** Core experiments, logs, figures, documentation, and known limitations will be frozen for final review.

8.7 Phase 1 Completion Criteria

Phase 1 will be considered complete when the team can demonstrate the following outcomes:

- The common workflow runs from document upload through playable audio using documented setup instructions.
- Each major stage produces a structured artifact that can be inspected and tested independently.
- The generated script contains evidence references for substantive factual claims, and unresolved claims are revised, qualified, or rejected before rendering.
- At least one representative document can be processed without manual rewriting of the final script in the common path.
- The team has executed at least one baseline workflow and one PaperCast workflow and recorded their configurations and outputs.
- The repository contains a reproduction note, configuration examples without secrets, automated checks for core components, and a record of known limitations.

Chapter 9. Project Schedule

Sprint	Asha	Gayathri	Manasa	Sreya	Checkpoint
S1: Sep 24-30	Define listener modes and UX requirements	Finalize corpus and parser requirements	Complete literature matrix and evaluation rubric	Define agent state machine and schemas	Scope review
S2: Oct 1-14	Build upload and mode-selection screens	Implement parser, chunker, and metadata model	Set up baseline runner and experiment configurations	Develop planning and generation prompts	First text prototype
S3: Oct 15-28	Build script and evidence display	Implement vector index and hybrid retrieval	Run baseline experiments and initial metrics	Implement evidence-linked generation	Baseline and retrieval demo
S4: Oct 29-Nov 11	Add playback and duration controls	Run extraction and retrieval tests	Create claim annotations and reviewer rubric	Implement verification and revision loop	Verified script prototype
S5: Nov 12-25	Conduct usability review	Run robustness and parser tests	Run experiments and collect evaluation results	Integrate TTS and audio segment assembly	Integrated prototype
S6: Nov 26-Dec 9	Prepare demonstration flow	Prepare deployment tutorial and release packaging	Complete statistical analysis and results	Complete ablations, figures, and documentation	Experiment freeze
Final: Dec 10-16	Rehearse final demonstration	Support release packaging and deployment notes	Complete final report and references	Perform final QA, fixes, and archive	Final demonstration and project release